Digital Signature를 이용한 무결성 기능을 지원하는

파일 전송 SW 구현

2026.10.12.

팀 해시브라운 

강정우
이원준
최재훈

INDEX

목차

1. 프로젝트 목표

2. 계획 대비 진행상황

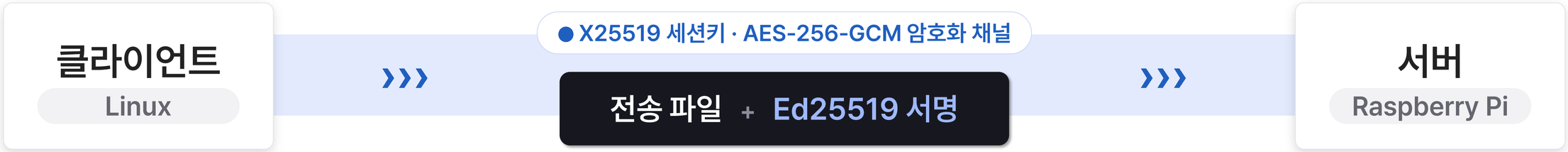
3. 역할 분담

4. 설계 사양

5. 연구 내용 및 결과물

6. 구현 결과 및 향후 계획

연구 목적



무결성 Ed25519 디지털 서명

파일 해시에 서명하고 수신 측이 공개키로 검증, 실패하면 저장을 거부

기밀성 AES-256-GCM

파일과 파일 목록을 암호화하고, 인증 태그로 변조까지 탐지

인증 X25519 키 교환

접속마다 새 세션키를 만들고, 등록된 키로 서버 · 클라이언트를 상호 확인

안전한 파일 전송 소프트웨어 구현

연구 일정

연구 내용	3월	4월	5월	6월	7월	8월	9월	10월	진행
관련 내용 학습	■	■	■						완료
개발 환경 구축		■							완료
파일전송, 디지털 서명 모듈 구현		■	■	■					완료
AES 암호화 모듈 구현					■	■			완료
기능 개선							■	■	완료
라즈베리파이 동작 시험 및 최종 점검								■	완료
작품전시회 준비								■	예정

연구 일정

연구 내용	3월	4월	5월	6월	7월	8월	9월	10월	진행
관련 내용 학습	■	■	■						완료
개발 환경 구축		■							완료
파일전송, 디지털 서명 모듈 구현		■	■	■					완료
AES 암호화 모듈 구현					■	■			완료
기능 개선							■	■	완료
라즈베리파이 동작 시험 및 최종 점검								■	완료
작품전시회 준비								■	예정

역할 분담

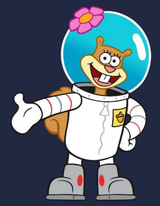
강정우

파일 전송 구현 및
OpenSSL 사용법 학습



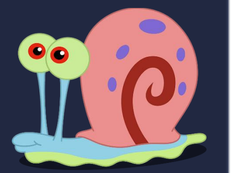
이원준

디지털 서명 및
무결성 검증 모듈 구현
(OpenSSL)

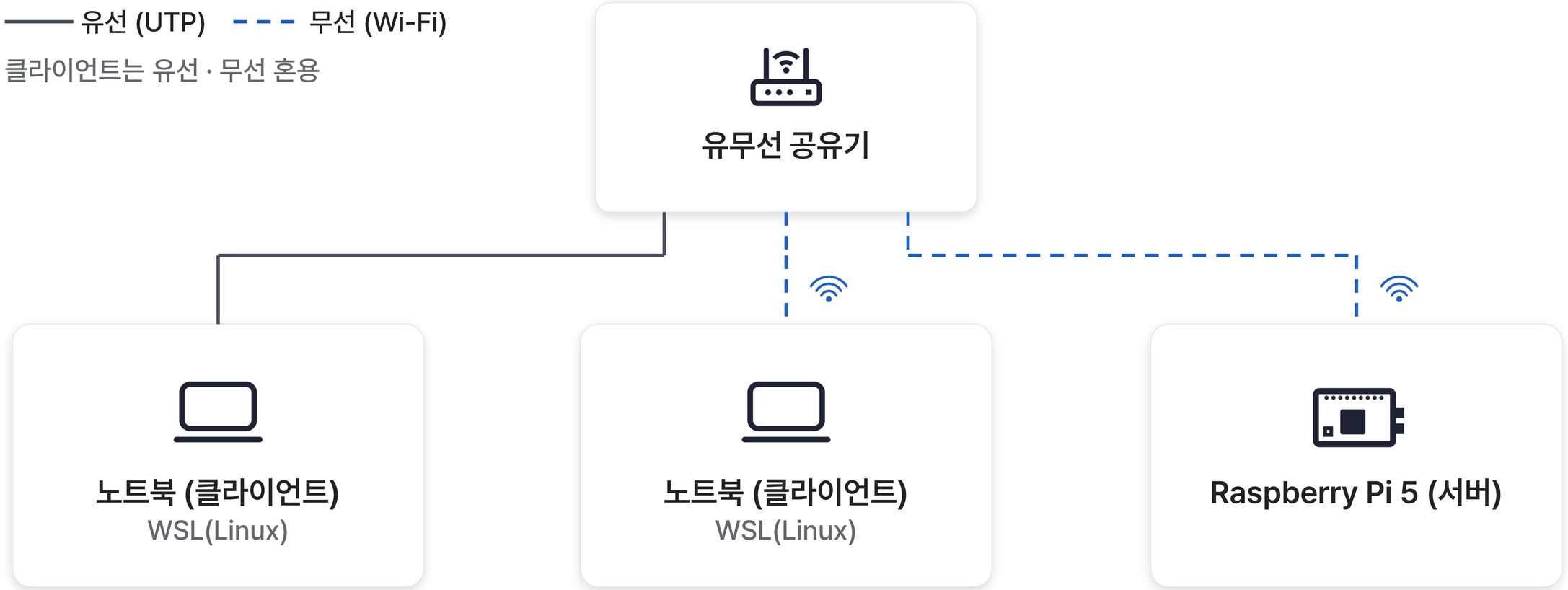


최재훈

개발 환경 구성
기능 통합 및 테스트
보고서 작성



설계 사양(최종 적용 환경)


유선 (UTP)
무선 (Wi-Fi)
클라이언트는 유선 · 무선 혼용
유무선 공유기
노트북 (클라이언트)
WSL(Linux)
노트북 (클라이언트)
WSL(Linux)
Raspberry Pi 5 (서버)

전체 동작 흐름

접속할 때마다

1. **접속**
   서버가 fork()로 전담 프로세스 생성
2. **상호 인증**
   Ed25519 서명으로 서버 · 클라이언트 확인
3. **세션키 생성**
   X25519 + HKDF → AES-256 키
4. **명령 처리**
   업로드 · 다운로드 · 목록 · 종료

파일 전송 (업로드 기준)

보내는 쪽 (클라이언트)

SHA-256
파일 해시 →
Ed25519 서명
이름 · 크기 · 해시 →
AES-GCM
암호화 · 64 KB 단위

전송
암호문 · 서명 · 태그

받는 쪽 (서버)

태그 검증
변조 시 중단 →
해시 비교
다시 계산해 비교 →
서명 검증
송신자 공개키 →
저장
모두 통과 시

※ 다운로드는 방향만 반대 (서버가 보내고 클라이언트가 검증)

계획 발표 대비 개선 사항

항목	계획 발표 (9. 7.)	현재 (개선 · 추가)
세션키	aes_key.bin을 양쪽에 미리 복사	X25519 키 교환 → 접속마다 새 AES-256 세션키
접속 인증	없음 (파일 서명만 검증)	키 교환 시 Ed25519로 서버 · 클라이언트 상호 인증
클라이언트 키	키 1쌍, 서버는 client_public.pem 하나만 신뢰	서버 clients/의 *.pem으로 등록 · 식별
동시 접속	한 번에 1대	fork()로 여러 대 동시 접속 가능
사용 방식	up / down 입력 후 파일명 직접 입력	메뉴 · 서버/내 파일 목록의 파일을 번호로 선택

사전 준비 – 키 쌍 생성

서버

```
# 서버 비밀키 생성
openssl genpkey -algorithm ed25519 -out server_private.pem

# 비밀키에서 공개키 생성
openssl pkey -in server_private.pem -pubout -out server_public.pem
```



비밀키

server_private.pem

서버의 비밀키, 서버에만 보관



공개키

server_public.pem

서버의 공개키, 모든 클라이언트에 배포

클라이언트

```
# 클라이언트 비밀키 생성
openssl genpkey -algorithm ed25519 -out client_private.pem

# 비밀키에서 공개키 생성
openssl pkey -in client_private.pem -pubout -out client_public.pem
```



비밀키

client_private.pem

클라이언트 비밀키,
해당 클라이언트에만 보관



공개키

client_public.pem

클라이언트 공개키,
서버에 각 클라이언트의 공개키 보관

사전 준비 – 키 보관 및 등록

실제 서버 폴더 구조

```
~/capstonefinal/
├── server                서버 실행 파일
├── server_private.pem    서버 비밀키
├── clients/              등록된 클라이언트 공개키
│   ├── android3.pem      태블릿의 공개키
│   ├── pc1.pem           데스크톱의 공개키
│   └── pc2.pem           노트북의 공개키
└── Files/                업로드 파일 저장

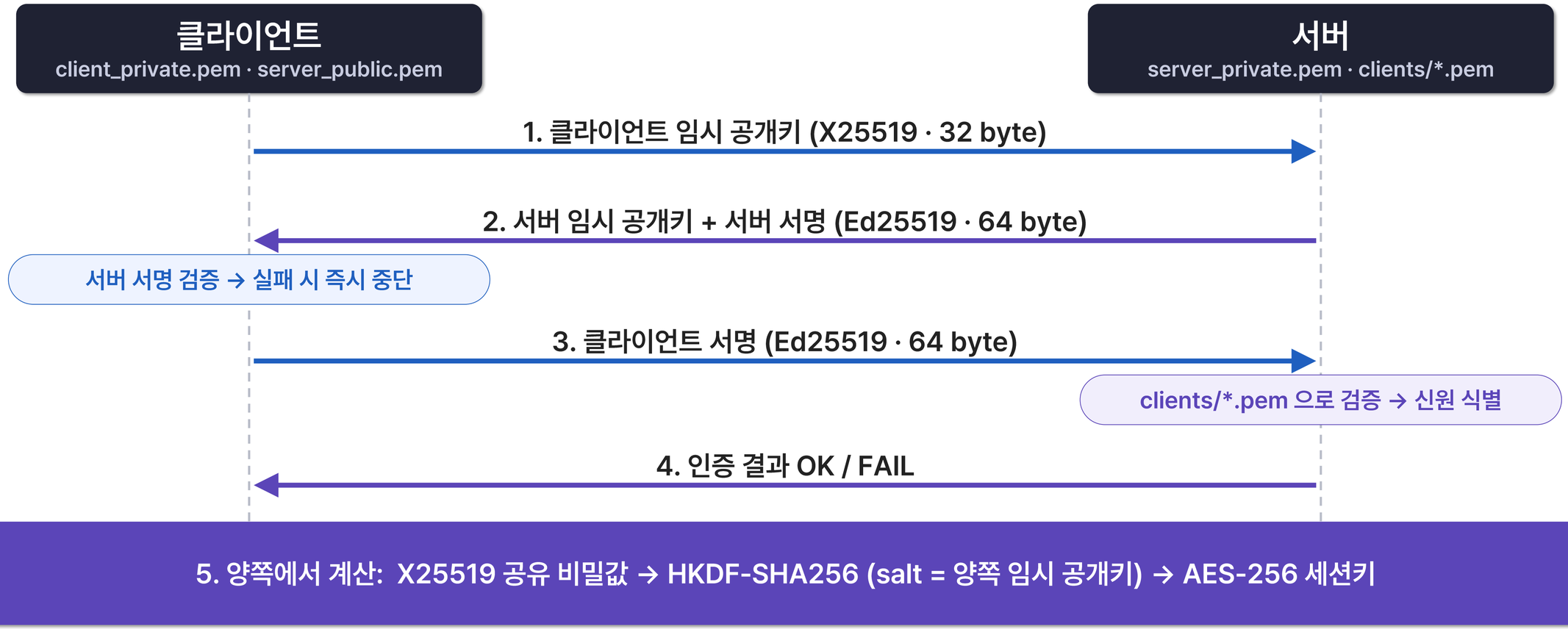
# 각 클라이언트의 공개키를 식별 가능한 이름으로 변경 후 복사
```

1. **서버 시작**
   clients/의 .pem 파일을 읽어 등록 목록 표시
2. **접속할 때마다 서명 확인**
   클라이언트 서명을 등록된 공개키로 하나씩 검증, 맞는 키가 없으면 연결 종료
3. **맞는 키 = 클라이언트 신원**
   파일 이름이 곧 클라이언트 이름, 업로드 서명도 이 키로 검증
4. **등록 추가 · 취소**
   파일을 넣거나 지우면 다음 접속부터 반영, 서버 재시작 불필요

시연 당시 서버 실제 출력

```
등록된 클라이언트 공개키 (clients/): android3 pc1 pc2 (3개)
[X25519 키 교환] 192.168.174.19:48468 (PID 8124)
  클라이언트 서명 검증(Ed25519): 성공 -> 인증된 클라이언트: android3 (clients/android3.pem)
```

X25519 키 교환 및 상호 인증



X25519 세션키 유도

client.c

```
// 1. 접속마다 새 임시 키 쌍 → 공개키 전송
client_key = x25519_generate_key(client_pub);
send_all(sock, client_pub, 32);

// 2. 서버 서명 검증 (실패 = 가짜 서버 → 중단)
make_handshake_info(&hs, "X25519-SERVER", ...);
eddsa_verify_with_key(&hs, ..., SERVER_PUBLIC_KEY);

// 3. 내 서명 전송 → 서버가 등록 키로 신원 확인
eddsa_sign_with_key(&hs, ..., CLIENT_PRIVATE_KEY);
```

x25519.c

```
// 공유 비밀값 = 내 임시 개인키 × 상대 임시 공개키
EVP_PKEY_derive_set_peer(dh_ctx, peer);
EVP_PKEY_derive(dh_ctx, shared, &shared_len);

// HKDF-SHA256 (salt = 양쪽 임시 공개키 64 byte)
EVP_PKEY_CTX_set_hkdf_md(kdf_ctx, EVP_sha256());
EVP_PKEY_CTX_set1_hkdf_salt(kdf_ctx, salt, salt_len);
EVP_PKEY_derive(kdf_ctx, session_key, &key_len);

OPENSSL_cleanse(shared, sizeof(shared));
```

1. 임시 키 생성
접속마다 새 X25519 키

→

2. 서명 교환 · 검증
Ed25519 상호 인증

→

3. 공유 비밀값
X25519 (ECDH)

→

4. 세션키 유도
HKDF-SHA256 → 32 byte

다중 클라이언트 동시 접속 – fork()

accept()
부모 프로세스가 접속 대기

↓

fork()
접속 1건마다 자식 프로세스 생성

↓

자식 프로세스
키 교환 · 명령 처리 · 종료 시 키 소거

↓

부모 프로세스
소켓을 넘기고 바로 다음 접속 대기

↓

동시 업로드 충돌 방지
임시 파일 이름.<PID>.tmp → 검증 후 rename

server.c

```c
signal(SIGCHLD, SIG_IGN);      // 끝난 자식 자동 회수

while (1) {
    client_sock = accept(server_sock, ...);
    pid_t pid = fork();
    if (pid == 0) {            // 자식: 클라이언트 1명 담당
        close(server_sock);
        handle_client(client_sock);
        OPENSSL_cleanse(aes_key, sizeof(aes_key));
        exit(0);
    }
    close(client_sock);        // 부모: 다음 접속 대기
}
```

메뉴 및 파일 목록 조회

클라이언트

```
========== 메뉴 ==========
 1. 업로드          (up)
 2. 다운로드        (down)
 3. 서버 파일 목록  (slist)
 4. 내 파일 목록    (clist)
 0. 종료            (exit)
==========================
번호 또는 명령어 입력: 2

[서버 파일 목록] 4개 (AES-256-GCM 복호화/인증: 성공)
  번호  파일명                 크기(byte)
     1  dataset.csv             80000000
     2  lecture_video.mp4      200000000
     3  server_manual.pdf        1234567
     4  공지사항.txt                2048
다운로드할 파일 번호를 입력하세요 (1~4, 취소: 0): 2
```

1 번호로 선택하는 메뉴

1 업로드 · 2 다운로드 · 3 서버 목록 · 4 내 목록 · 0 종료,
파일도 목록 번호로 선택 가능

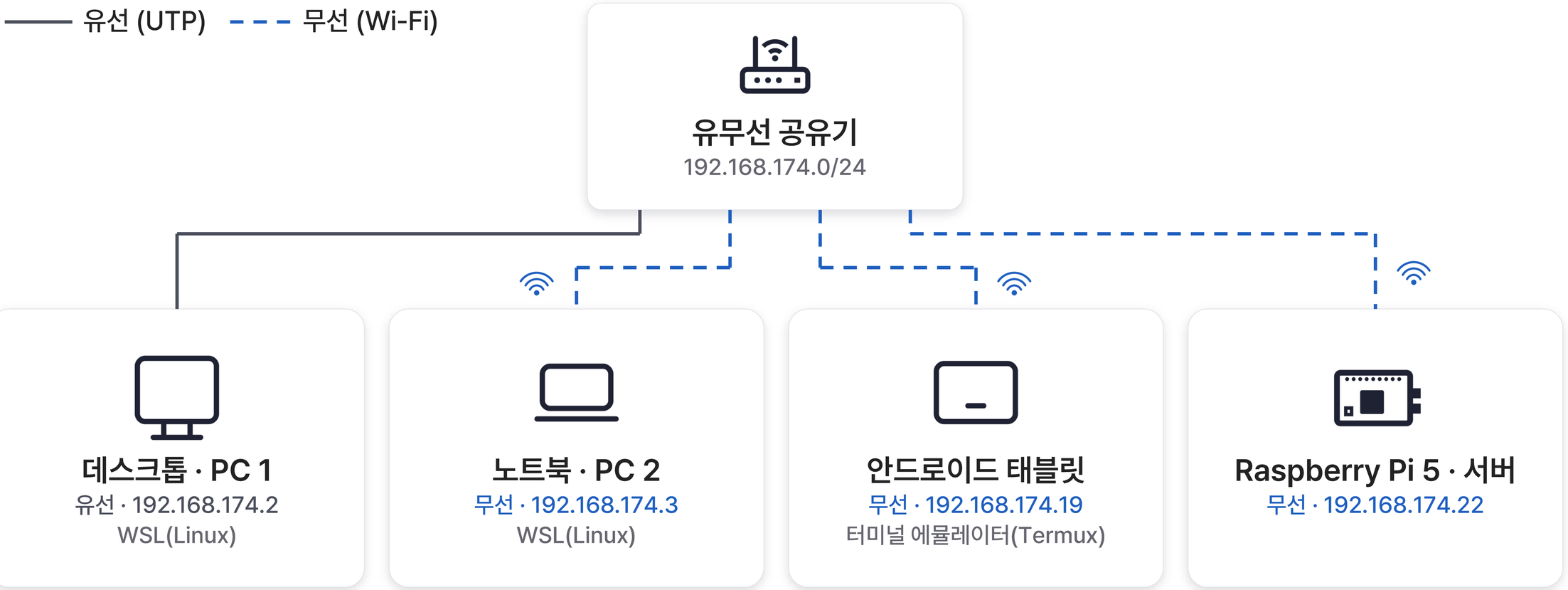
2 서버 파일 목록도 암호화하여 전송

서버 파일 목록을 AES-256-GCM으로 암호화,
헤더(개수 · IV)는 AAD로 묶어 조작 탐지

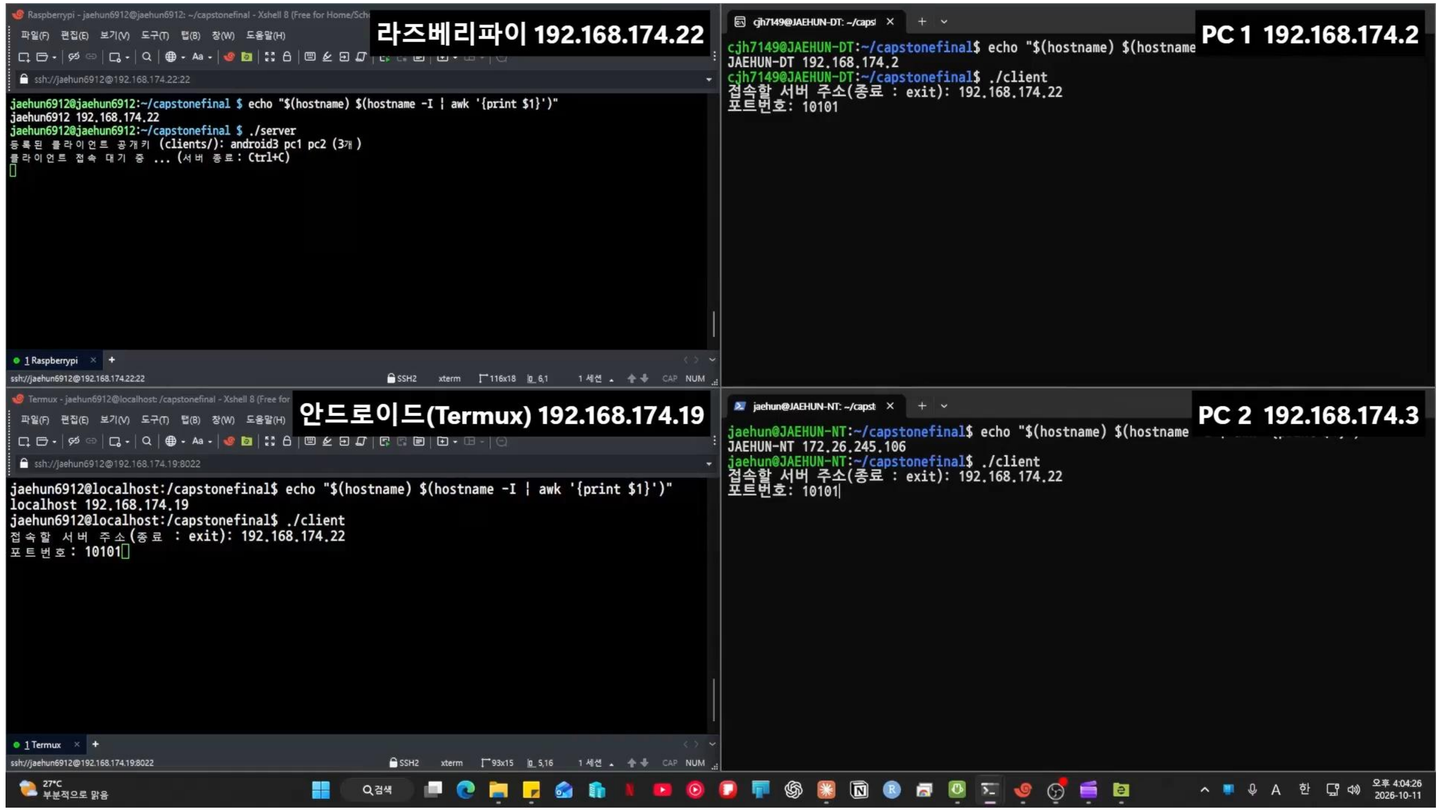
3 안전한 파일명 처리

경로 부분 제거, 잘못된 이름 거부,
전송 중 임시 파일은 목록에서 제외

시연 환경


유선 (UTP)
무선 (Wi-Fi)
유무선 공유기
192.168.174.0/24
데스크톱 · PC 1
유선 · 192.168.174.2
WSL(Linux)
노트북 · PC 2
무선 · 192.168.174.3
WSL(Linux)
안드로이드 태블릿
무선 · 192.168.174.19
터미널 에뮬레이터(Termux)
Raspberry Pi 5 · 서버
무선 · 192.168.174.22

시연 영상


라즈베리파이 192.168.174.22
jaehun6912@jaehun6912:~/capstonefinal $ echo "$(hostname) $(hostname -I | awk '{print $1}')"
jaehun6912 192.168.174.22
jaehun6912@jaehun6912:~/capstonefinal $ ./server
등록된 클라이언트 공개키 (clients/): android3 pc1 pc2 (3개)
클라이언트 접속 대기 중 ... (서버 종료: Ctrl+C)
PC 1 192.168.174.2
cjh7149@JAEHUN-DT:~/capstonefinal$ echo "$(hostname) $(hostname
JAEHUN-DT 192.168.174.2
cjh7149@JAEHUN-DT:~/capstonefinal$ ./client
접속할 서버 주소(종료 : exit): 192.168.174.22
포트번호: 10101
안드로이드(Termux) 192.168.174.19
jaehun6912@localhost:/capstonefinal$ echo "$(hostname) $(hostname -I | awk '{print $1}')"
localhost 192.168.174.19
jaehun6912@localhost:/capstonefinal$ ./client
접속할 서버 주소(종료 : exit): 192.168.174.22
포트번호: 10101
PC 2 192.168.174.3
jaehun@JAEHUN-NT:~/capstonefinal$ echo "$(hostname) $(hostname
JAEHUN-NT 172.26.245.106
jaehun@JAEHUN-NT:~/capstonefinal$ ./client
접속할 서버 주소(종료 : exit): 192.168.174.22
포트번호: 10101

구현 결과 및 향후 계획

구현 결과

무결성
Ed25519 서명 + SHA-256, 변조 · 위조 파일은 저장 거부

기밀성
AES-256-GCM으로 파일 · 목록 암호화 전송

인증 · 키 관리
X25519 세션키 + 클라이언트별 키 등록

실사용 환경
Raspberry Pi 5 서버에 데스크톱 · 노트북 · 태블릿 동시 접속

향후 계획

- 작품전시회 준비
- 추가 개선사항 식별

감사합니다

팀 해시브라운 